Java w IntelliJ IDEA - teksty, klasy i metody

BLOK 5 - String, obiekty, hermetyzacja, konstruktory

1. Co można zrobić z napisem String?

`String` reprezentuje napis, czyli sekwencję znaków. Przykładem jest imię, tytuł książki albo adres e-mail. W przeciwieństwie do typu `char`, który reprezentuje pojedynczą jednostkę kodową, napis może zawierać wiele znaków. Napis zapisujemy w podwójnych cudzysłowach, np. `"Ala"`. Możemy sprawdzić jego długość, porównać z innym napisem albo pobrać fragment.

```
String text = "Java";
System.out.println(text.length());
System.out.println(text.toUpperCase());
System.out.println(text.charAt(0));
```

```
Wynik:
4
JAVA
J
```

`length()` zwraca długość napisu w jednostkach UTF-16. `toUpperCase()` tworzy wersję z wielkimi literami. `charAt(0)` zwraca pierwszy znak, ponieważ liczenie pozycji rozpoczyna się od zera. W napisie `"Java"` indeksy to 0, 1, 2 i 3; próba użycia `charAt(4)` spowoduje błąd podczas wykonania.

2. String nie zmienia własnej zawartości

Obiekty klasy `String` są niemutowalne: po utworzeniu nie da się zmienić znaków wewnątrz tego samego obiektu. Metody takie jak `replace`, `trim` czy `toLowerCase` zwracają nowy napis. Jeżeli nie zapiszemy wyniku do zmiennej, stara wartość pozostanie niezmieniona.

```
String name = "  Ola  ";
name.trim();
System.out.println("[" + name + "]");
name = name.trim();
System.out.println("[" + name + "]");
```

```
Wynik:
[  Ola  ]
[Ola]
```

Pierwsza instrukcja `trim()` oblicza nowy tekst, ale nigdzie go nie zapisuje. Dopiero `name = name.trim()` przypisuje wynik do zmiennej. Nie oznacza to modyfikacji starego obiektu: zmienna `name` zaczyna wskazywać na inny napis. To ważny przykład różnicy między zmienną, która może zmienić przypisanie, a samym obiektem.

3. Łączenie napisów i metoda equals

Operator `+` łączy napisy. Jeżeli jedną stroną jest `String`, Java może przekształcić inną wartość do reprezentacji tekstowej. Do porównania treści napisów używamy `equals`, ponieważ `==` dla obiektów oznacza tożsamość referencji. Wielkość liter ma znaczenie dla zwykłego `equals`.

```
String city = "Kraków";
int population = 800000;
System.out.println("Miasto: " + city);
System.out.println("Ludność: " + population);
System.out.println(city.equals("kraków"));
```

```
Wynik:
Miasto: Kraków
Ludność: 800000
false
```

W ostatniej instrukcji napis z małą literą `k` nie jest równy napisowi z wielką literą `K`. Jeżeli zależy nam na porównaniu bez rozróżniania wielkości liter, istnieje `equalsIgnoreCase`. Należy jednak wiedzieć, jakie dokładnie porównanie jest potrzebne w zadaniu.

4. Klasa jako przepis na tworzenie obiektów

Klasa opisuje, jakie informacje i zachowania mogą mieć obiekty określonego rodzaju. Można porównać ją do formularza opisującego ucznia: każde dziecko ma imię i wiek, ale poszczególni uczniowie mają różne dane. Definicja klasy nie jest jeszcze konkretnym obiektem. Obiekt powstaje dopiero po utworzeniu instancji, zazwyczaj przez operator `new`.

```
class Student {
    String name;
    int age;
}

Student first = new Student();
first.name = "Maja";
first.age = 20;
```

`class Student` definiuje nowy typ obiektowy. `String name` i `int age` są polami: każdy utworzony obiekt `Student` posiada własne wartości tych pól. `new Student()` tworzy obiekt. Zmienna `first` nie jest tym obiektem, lecz przechowuje referencję pozwalającą się do niego odwoływać. Zapis `first.name` oznacza dostęp do pola `name` tego konkretnego obiektu.

5. Metoda - zachowanie przypisane do klasy lub obiektu

Metoda to fragment programu wykonujący określone zadanie. Można przekazać jej parametry, a ona może zwrócić wynik. Dzięki temu nie kopiujemy tej samej logiki w wielu miejscach. W prostym przykładzie metoda oblicza pole prostokąta na podstawie długości boków.

```
static int area(int width, int height) {
    return width * height;
}

System.out.println(area(3, 4));
```

```
Wynik:
12
```

`int` przed nazwą metody informuje, że metoda zwraca liczbę całkowitą. W nawiasach znajdują się dwa parametry: `width` i `height`. `return width * height` kończy działanie metody i przekazuje wynik do miejsca jej wywołania. Zapis `area(3, 4)` oznacza, że w tym wywołaniu szerokość wynosi 3, a wysokość 4. W pełnym programie metoda powinna znajdować się wewnątrz klasy.

6. Czym różni się parametr od argumentu?

Parametr jest nazwą zmiennej podawaną w deklaracji metody, np. `int width`. Argument to konkretna wartość użyta przy wywołaniu, np. liczba 3 w `area(3, 4)`. Metoda otrzymuje wartości argumentów do własnych parametrów. Dwie różne instrukcje mogą wywoływać tę samą metodę z innymi danymi, dlatego metoda jest uniwersalna.

```
static String greet(String person) {
    return "Cześć, " + person;
}

System.out.println(greet("Ania"));
System.out.println(greet("Bartek"));
```

```
Wynik:
Cześć, Ania
Cześć, Bartek
```

Nazwa parametru `person` istnieje w obrębie metody. Argumentami są odpowiednio napisy `"Ania"` i `"Bartek"`. Dwa wywołania produkują różne wyniki bez zmieniania definicji metody. Gdy metoda zwraca `String`, możemy jej wynik od razu wyświetlić lub zapisać w zmiennej.

7. Pola i modyfikatory dostępu

Pole to zmienna związana z klasą albo obiektem. Jeżeli oznaczymy je jako `public`, dostęp do niego może być możliwy z innych klas. `private` ogranicza bezpośredni dostęp do wnętrza klasy. Często jest to korzystne, ponieważ kod z zewnątrz nie powinien móc dowolnie psuć poprawnego stanu obiektu.

```
class Account {
    private double balance = 0;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}
```

W przykładzie pole `balance` jest prywatne, ale metoda `deposit` jest publiczna. Użytkownik klasy nie może więc bezpośrednio ustawić ujemnego salda, lecz może wywołać metodę, która najpierw sprawdza kwotę. Jest to pierwszy krok do **hermetyzacji**, czyli ukrywania szczegółów danych za dobrze określonymi metodami.

Składnik	Do czego służy?	Przykład

pole	**przechowuje stan obiektu**	`private int age;`
konstruktor	ustawia stan przy tworzeniu obiektu	`Person(String name)`
getter	odczytuje wartość	`getAge()`
setter	umożliwia kontrolowaną zmianę	`setAge(20)`

8. Gettery i settery - kontrolowany odczyt oraz zmiana

Getter zwykle zwraca wartość pola. Setter przyjmuje nową wartość i może zdecydować, czy wolno ją zapisać. Nie każda klasa musi mieć setter dla każdego pola. Jeżeli np. numer identyfikacyjny nie powinien się zmieniać po utworzeniu obiektu, nie musimy dodawać metody do jego modyfikacji.

```
class Product {
    private double price;

    public double getPrice() { return price; }

    public void setPrice(double newPrice) {
        if (newPrice >= 0) {
            price = newPrice;
        }
    }
}
```

Jeżeli wywołamy `setPrice(20)`, pole przyjmie 20. Przy `setPrice(-5)` metoda nie zapisze wartości, bo warunek jest fałszywy. To najprostsza forma walidacji. W bardziej rozbudowanym programie można zgłosić wyjątek zamiast po cichu odrzucać nieprawidłowe dane, ale zasada pozostaje ta sama: obiekt sam pilnuje swojego stanu.

9. Konstruktor - nadanie danych początkowych obiektowi

Konstruktor wykonuje się podczas tworzenia obiektu. Ma taką samą nazwę jak klasa i nie podaje się dla niego typu zwracanego. Pozwala od razu nadać polom odpowiednie wartości, zamiast tworzyć obiekt z przypadkowymi danymi i dopiero potem go uzupełniać.

```
class Book {
    String title;

    Book(String initialTitle) {
        title = initialTitle;
    }
}

Book book = new Book("Mały Książę");
System.out.println(book.title);
```

```
Wynik:
Mały Książę
```

Napis przekazany do `new Book(...)` staje się argumentem konstruktora. W jego wnętrzu pole `title` otrzymuje początkową wartość. Konstruktor nie jest wywoływany tak samo jak zwykła metoda zwracająca `Book`; jego zadaniem jest zainicjalizowanie właśnie tworzonego obiektu.

10. this - wskazanie konkretnego obiektu

Słowo `this` odnosi się do aktualnego obiektu, dla którego wykonywana jest metoda lub konstruktor instancji. Przydaje się zwłaszcza wtedy, gdy parametr ma taką samą nazwę jak pole. Bez `this` nazwa w danym miejscu wskazywałaby parametr, zasłaniając pole.

```
class Person {
    private String name;

    Person(String name) {
        this.name = name;
    }
}
```

Po lewej stronie `this.name` oznacza pole tworzonego obiektu. Po prawej stronie `name` oznacza parametr konstruktora. Instrukcja przypisuje więc otrzymany argument do pola. Bez `this`, przy identycznych nazwach, łatwo napisać błędne `name = name`, które nie zainicjalizuje pola zgodnie z zamiarem.

11. Przeciążanie metod i konstruktorów

Przeciążanie oznacza istnienie w jednej klasie kilku metod o tej samej nazwie, ale różnych listach parametrów. Java wybiera odpowiednią wersję na podstawie liczby i typów argumentów. Sam inny typ zwracany nie wystarcza do poprawnego przeciążenia.

```
static int add(int a, int b) {
    return a + b;
}
static double add(double a, double b) {
    return a + b;
}
```

Wywołanie `add(2, 3)` korzysta z wersji dla `int`, natomiast `add(2.5, 3.5)` z wersji dla `double`. Podobnie konstruktor może mieć wersję przyjmującą imię i wiek oraz prostszą wersję bez argumentów. Dzięki przeciążaniu jedna czynność może być używana w kilku wygodnych wariantach.

12. Składowe statyczne - co nie należy do konkretnego obiektu?

Metoda lub pole oznaczone `static` należy do klasy, a nie do pojedynczej instancji. Przykładem jest metoda `Math.max`, którą wywołujemy jako `Math.max(3, 5)`, bez wcześniejszego tworzenia obiektu `Math`. Pojęcie `static` zostanie rozwinięte w następnym temacie; tutaj warto je rozpoznać w zapisie metody.

```
int bigger = Math.max(3, 5);
System.out.println(bigger);
```

```
Wynik:
5
```

W praktyce możemy pisać własne metody statyczne wykonujące obliczenia niezależne od stanu obiektu. Jeżeli metoda ma korzystać z pól konkretnego ucznia lub konta, zwykle powinna być metodą instancji.

Przykład 1. Example01StringOperations

Wykorzystujemy metody `String`. Szczególnie ważne jest rozróżnienie metody zwracającej nowy napis i zmiany wartości zmiennej.

```
package block5;

public class Example01StringOperations {
    public static void main(String[] args) {
        String text = "  Java Programming  ";
        String clean = text.trim();

        System.out.println(clean.length());
        System.out.println(clean.toUpperCase());
        System.out.println(clean.contains("Program"));
        System.out.println(clean.substring(0, 4));
        System.out.println(clean.replace("Java", "Kotlin"));
        System.out.println(clean.equals("Java Programming"));
    }
}
```

Wynik:

```
16
JAVA PROGRAMMING
true
Java
Kotlin Programming
true
```

- Zapis `String text = "  Java Programming  ";` tworzy zmienną `text` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `String clean = text.trim();` tworzy zmienną `clean` typu `String` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(clean.length());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

- Instrukcja `System.out.println(clean.toUpperCase());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(clean.contains("Program"));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(clean.substring(0, 4));` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 2. Example02ClassAndObject

Tworzymy obiekt z klasy i odwołujemy się do jego danych oraz zachowań.

```
package block5;

public class Example02ClassAndObject {
    public static void main(String[] args) {
        Person person = new Person("Anna", 21);
        person.introduce();

        person.setAge(22);
        System.out.println(person.getName() + " ma teraz " + person.getAge() + " lata.");
    }
}
```

Wynik:

```
Mam na imię Anna i mam 21 lat.
Anna ma teraz 22 lata.
```

- W zapisie `Person person = new Person("Anna", 21);` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(person.getName() + " ma teraz " + person.getAge() + " lata.");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 3. Person

Odczytujemy przykład klasy z polami prywatnymi, konstruktorem, metodami dostępu i sprawdzaniem wieku.

```
package block5;

public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        setAge(age);
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public final void setAge(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("Wiek nie może być ujemny");
        }
        this.age = age;
    }

    public void introduce() {
        System.out.println("Mam na imię " + name + " i mam " + age + " lat.");
    }
}
```

- Instrukcja `return name;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `return age;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- W zapisie `if (age < 0) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Wiersz `throw new IllegalArgumentException("Wiek nie może być ujemny");` świadomie zgłasza wyjątek, sygnalizując problem zamiast kontynuować zwykłe wykonanie.
- Instrukcja `System.out.println("Mam na imię " + name + " i mam " + age + " lat.");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. Example03Overloading

Wybieramy przeciążoną wersję metody na podstawie typów przekazanych argumentów.

```
package block5;

public class Example03Overloading {
    public static void main(String[] args) {
        Rectangle unit = new Rectangle();
        Rectangle square = new Rectangle(5);
        Rectangle rectangle = new Rectangle(4, 7);

        System.out.println(unit.area());
        System.out.println(square.area());
        System.out.println(rectangle.area());

        MathHelper helper = new MathHelper();
        System.out.println(helper.max(2, 9));
        System.out.println(helper.max(2.5, 1.8));
        System.out.println(helper.max(4, 9, 3));
    }
}
```

Wynik:

```
1.0
25.0
28.0
9
2.5
9
```

- W zapisie `Rectangle unit = new Rectangle();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- W zapisie `Rectangle square = new Rectangle(5);` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- W zapisie `Rectangle rectangle = new Rectangle(4, 7);` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(unit.area());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(square.area());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(rectangle.area());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Zadanie 5. Product

Obserwujemy zależność między stanem obiektu a metodami, które go udostępniają lub modyfikują.

Polecenie: Prywatne pola: name, price. Zrób dwa konstruktory: Product(String name) z ceną 0 oraz Product(String name, double price). Dodaj gettery i setter ceny, który odrzuci wartość ujemną.

```
package block5;

public class Product {
    // TODO
}
```

Przykład 6. BankAccount

Analizujemy przykład konta, którego saldo powinno być zmieniane przez kontrolowane metody.

```
package block5;

public class BankAccount {
    private final String owner;
    private double balance;

    public BankAccount(String owner) {
        this(owner, 0.0);
    }

    public BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Saldo początkowe nie może być ujemne");
        }
        this.balance = initialBalance;
    }

    public String getOwner() {
        return owner;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Wpłata musi być dodatnia");
        }
        balance += amount;
    }

    public boolean withdraw(double amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }
}
```

- W zapisie `if (initialBalance < 0) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Wiersz `throw new IllegalArgumentException("Saldo początkowe nie może być ujemne");` świadomie zgłasza wyjątek, sygnalizując problem zamiast kontynuować zwykłe wykonanie.
- Instrukcja `return owner;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.

- Instrukcja `return balance;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- W zapisie `if (amount <= 0) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Wiersz `throw new IllegalArgumentException("Wpłata musi być dodatnia");` świadomie zgłasza wyjątek, sygnalizując problem zamiast kontynuować zwykłe wykonanie.

Przykład 7. Rectangle

Klasa `Rectangle` ilustruje kolejny praktyczny wariant pojęć opisanych w tym bloku. Przyjrzyjmy się instrukcjom i powiązaniu wartości z wynikiem.

```
package block5;

public class Rectangle {
    private double width;
    private double height;

    public Rectangle() {
        this(1.0, 1.0);
    }

    public Rectangle(double side) {
        this(side, side);
    }

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double area() {
        return width * height;
    }
}
```

- Instrukcja `return width * height;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.

Przykład 8. MathHelper

Klasa `MathHelper` ilustruje kolejny praktyczny wariant pojęć opisanych w tym bloku. Przyjrzyjmy się instrukcjom i powiązaniu wartości z wynikiem.

```
package block5;

public class MathHelper {
    public int max(int a, int b) {
        return a > b ? a : b;
    }

    public double max(double a, double b) {
        return a > b ? a : b;
    }

    public int max(int a, int b, int c) {
        return max(max(a, b), c);
    }
}
```

- Instrukcja `return a > b ? a : b;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `return max(max(a, b), c);` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.

13. Zadania - String i tworzenie klas

Zadanie 6. Utwórz napis `" Java "`, usuń z niego spacje z początku i końca, a następnie wyświetl liczbę znaków przed i po zmianie. Wyjaśnij, dlaczego samo wywołanie `trim()` nie zmieni pierwotnej wartości zmiennej.

Zadanie 7. Napisz klasę `Rectangle` z polami `width` i `height`. Dodaj konstruktor, który przyjmuje oba wymiary, oraz metodę `area()`. Utwórz dwa prostokąty i oblicz ich pola.

Zadanie 8. Napisz klasę `BankAccount` z prywatnym polem salda. Dodaj metodę wpłaty, która ignoruje lub odrzuca kwoty ujemne. Sprawdź, że kod z innej klasy nie może bezpośrednio zmienić pola prywatnego.

Zadanie 9. Przygotuj przeciążone metody `multiply(int, int)` i `multiply(double, double)`. Przeanalizuj, która wersja zostaje wybrana dla wywołań `multiply(2, 3)` i `multiply(2.0, 3.0)`.

Zadanie 10. Wyjaśnij, czy następujące zapisy dotyczą tego samego obiektu: `Book first = new Book("A"); Book second = new Book("A");`. Czym różni się porównywanie treści napisów od porównywania referencji do obiektów?